

WayMeet Backend — Cloud Deployment Plans

Go modular monolith + worker · Postgres/PostGIS · Redis · S3 storage | Goal: trigger the API from anywhere, low budget | 2026-06-23

Tech stack

Layer	Technology
Language / runtime	Go 1.26 — modular monolith (cmd/api) + always-on worker (cmd/worker)
HTTP	stdlib net/http (ServeMux), port 8080
Database	PostgreSQL 16 + PostGIS (geospatial "nearby people") — requires the PostGIS extension
DB access	sqlc + pgx/v5, golang-migrate
Cache / queue / realtime	Redis (chat pub/sub, cache, asynq jobs) + WebSockets
Object storage	S3-compatible (MinIO locally) — presigned uploads for images/video/voice
Auth	JWT access/refresh (+ planned OAuth)
External SaaS	Firebase FCM (push) · Resend (email OTP) — kept as-is
Packaging	Multi-stage Docker → distroless static image

Must be hosted: api container · worker container (always-on) · Postgres+PostGIS · Redis · S3 bucket. The PostGIS extension and the always-on worker are what rule out the cheapest pure-serverless shortcuts.

Plan comparison

	Plan A — Single VPS cheapest	Plan B — Managed PaaS recommended	Plan C — Google Cloud scalable
Idea	One small server running your existing docker-compose + api/worker	Push 2 container images; rent managed Postgres + Redis + storage	Cloud-native on GCP (you already use Firebase)
Compute	Hetzner / DigitalOcean / Vultr VPS (2 vCPU / 4 GB)	Railway or Render (api + worker from same Dockerfile)	Cloud Run: api scales to zero; worker min-instances=1
Postgres+PostGIS	Your postgis/postgis container	Neon or Supabase (both ship PostGIS)	Cloud SQL (or Neon to cut cost)
Redis	redis container	Upstash (free tier) or platform add-on	Upstash (Memorystore is pricey)
Storage	MinIO container	Cloudflare R2 (no egress fees) or Supabase Storage	GCS (S3-interop) or R2
HTTPS / domain	Caddy reverse proxy (auto Let's Encrypt)	Built in	Built in
Est. cost / month	\$5 – 12	\$15 – 35	\$25 – 50
Ops effort	High (you own backups, updates, uptime)	Low (managed backups, logs, deploy on push)	Medium (managed, but steeper setup)
Scaling	Manual; single point of failure	Easy, no rebuild	Auto-scaling api
Pros	Dirt cheap; zero code changes; identical to local	Best budget/ops balance; near free-tier; env-var swaps only	Auto-scale; tight Firebase/FCM integration; production-grade

	Plan A — Single VPS cheapest	Plan B — Managed PaaS recommended	Plan C — Google Cloud scalable
Cons	You run everything; no redundancy	More moving parts; cost creeps past free tiers	Priciest; always-on worker + Memorystore are cost traps
Best when	You just want it reachable at lowest cost	Low budget + low ops + room to grow	Traffic justifies autoscaling

Recommendation: Start with **Plan B** (Railway/Render + Neon or Supabase + Upstash + Cloudflare R2). It stays near free-tier cost, needs **no code changes** — only env-var swaps for DATABASE_URL, REDIS_URL, and S3_* — gives HTTPS API access from anywhere, and handles the always-on worker cleanly. Drop to **Plan A** to squeeze cost to ~\$5; move to **Plan C** when traffic demands autoscaling.

Before you migrate

- Your code uses the **AWS S3 SDK**, so any S3-compatible target (R2, Supabase, MinIO) works by setting S3_ENDPOINT + keys — no rewrite.
- Confirm the Postgres host allows CREATE EXTENSION postgis; — Neon, Supabase, and Cloud SQL all do; many bargain hosts do not.
- Build the worker image with --build-arg CMD=worker; the api uses the default CMD=api.
- Keep Firebase FCM and Resend as external SaaS — only their credentials/keys move to the host's secret store.